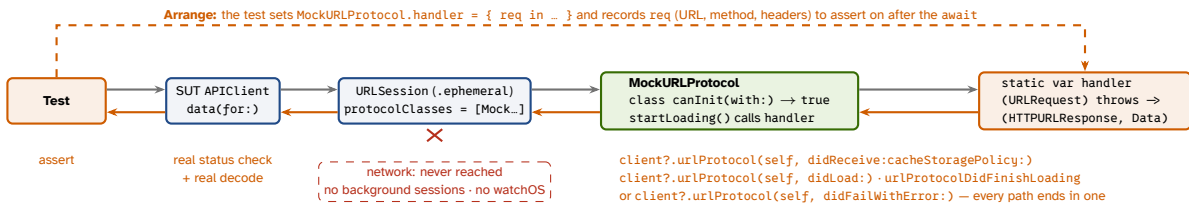


In one line: Wait for the **event**, never for **time**. async code: an `async` throws test that awaits. Callbacks: `XCTestExpectation + await fulfillment(of:)`, or Swift Testing's confirmation (which does *not* wait) or a continuation. Network: the *real* `URLSession` over a stub `URLProtocol`. Time: inject a `Clock` or scheduler and *advance* it.



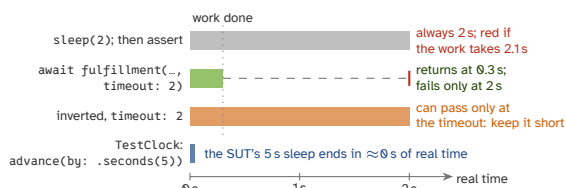
Pick the wait

need	XCTest	Swift Testing
async API	<code>func test() async throws</code> — arbitrary actor; sync tests run on the main actor	<code>@Test func f() async throws</code> — always an arbitrary task
callback, N times	<code>expectation(description:)</code> , <code>expectedFulfillmentCount = N</code> , <code>await fulfillment(of: [e], timeout: 2)</code>	<code>await confirmation(expectedCount: N) { c in ... c() }</code> — all N before the closure returns
at least N must not happen	<code>assertForOverFulfill = false</code> , <code>isInverted = true</code> ; fails at once if fulfilled	<code>expectedCount: N...</code> (6.1), <code>expectedCount: 0</code>
throws	<code>XCTestAssertThrowsError</code> takes a sync autoclosure → <code>do { try await ... }</code> catch	<code>await #expect(throws: E.self) { try await ... }</code> returns the error (6.1)
hang guard	<code>executionTimeAllowance: 600s</code> default, rounded up to whole minutes, only with test timeouts on	<code>.timeLimit(.minutes(n))</code> : minutes only; shortest wins; per argument case
parallel	a class runs its tests one by one	all at once, one process ; .serialized orders one suite only

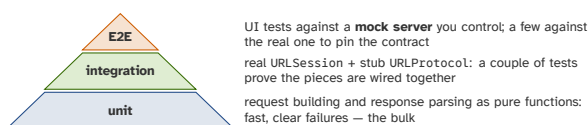
Expectations, precisely

- `wait(for:timeout:)` is **unavailable in async functions** since Xcode 14.3 (it can deadlock); `await fulfillment(of:timeout:)`. Left out, the timeout is `.infinity` — only `executionTimeAllowance` then ends a hang.
- A wait returns the moment all are fulfilled: the timeout is a *ceiling*. `XCTestWaiter.Result: completed · timedOut · incorrectOrder · invertedFulfillment · interrupted`.
- Over-fulfilling fails only with `assertForOverFulfill: true` from the `XCTestCase` convenience methods, false from `XCTestExpectation(description:)`.
- Predicate expectations **poll** — slow; prefer callback, KVO or notification ones (inject a private `NotificationCenter()`, hand it to the expectation).
- Swift Testing has no wait: a confirmation neither blocks nor suspends. `XCTestExpectation` is unsupported in `@Test` — even 6.4's interop (`XCTestAssert` now counts there) excludes it. Bridge a callback: `try await withCheckedThrowingContinuation { c in api.fetch(42) { c.resume(with: $0) } }` Resume **exactly once**: a second resume is a `FatalError`; none leaves the test suspended for ever and logs "leaked its continuation".

What each wait costs



Network tests in layers



On a device: Xcode's **Device Conditions** (Xcode 11) set a network-link profile (latency, loss, slow DNS) or a thermal state — the Network Link Conditioner, driven by Xcode.

Traps

- static var handler** under Swift Testing: all suites run in parallel in one process, and `.serialized` does not order unrelated suites — two suites overwrite one stub. Key handlers per test (own host or header → table) or nest every network suite in one `.serialized` parent. Swift 6 mode: `nonisolated(unsafe)` or a lock.
- `URLProtocol.registerClass` is global (consulted in reverse order) — set the session's `protocolClasses`. `.ephemeral` still keeps caches, cookies in RAM.

The stub

```
final class MockURLProtocol: URLProtocol {
    nonisolated(unsafe) static var handler:
        ((URLRequest) throws -> (HTTPURLResponse, Data))?
    override class func canInit(with r: URLRequest) -> Bool { true }
    override class func canonicalRequest(for r: URLRequest)
        -> URLRequest { r }
    override func startLoading() {
        do { let (resp, data) = try Self.handler!(request)
            client?.urlProtocol(self, didReceive: resp,
                cacheStoragePolicy: .notAllowed)
            client?.urlProtocol(self, didLoad: data)
            client?.urlProtocolDidFinishLoading(self)
        } catch { client?.urlProtocol(self, didFailWithError: error) } }
    override func stopLoading() {} }
    let config = URLSessionConfiguration.ephemeral
    config.protocolClasses = [MockURLProtocol.self]
    let sut = APIClient(session: URLSession(configuration: config))
    MockURLProtocol.handler = { req in (HTTPURLResponse(url: req.url!,
        statusCode: 500, httpVersion: nil, headerFields: nil)!, Data()) }
    await #expect(throws: APIError.status(500)) {
        try await sut.user(id: 42) } }
```

Four ways a request fails — stub each

path	the handler	what the client sees
transport	throw <code>URLError(.notConnectedToInternet)</code> (~100%) returns 404 / 500	<code>data(for:)</code> throws that <code>URLError</code> → map to .offline
HTTP status	returns 404 / 500	no throw (Data, URLResponse) comes back; you cast and check 200 ... 299
decoding	200 + malformed JSON	<code>DecodingError: keyNotFound · typeMismatch · valueNotFound · dataCorrupted</code>
cancelled	never finishes; the test cancels the Task	<code>URLError(.cancelled)</code> (~99%), <i>not</i> <code>CancellationError</code>

Assert the **request** too. Its body arrives as `httpBodyStream`; `httpBody` is often `nil` inside the protocol.

Time, streams, Combine

- Clock**: depend on any `Clock<Duration>` (iOS 16); `sleep(for:)` works on the existential since Swift 5.9. `ProdContinuousClock()`; tests (swift-clocks) `TestClock: await clock.advance(by: .seconds(5))`, `await clock.run()` until nothing sleeps; `ImmediateClock` never suspends; `UnimplementedClock` fails if used. Dates: `inject()` → `Date`. Timers / GCD: a scheduler protocol whose mock fires at once.
- Combine**: Apple's `ImmediateScheduler` ignores dates but has its own time types, so it cannot replace `DispatchQueue` without generics; combine-schedulers' `AnySchedulerOf<DispatchQueue> + DispatchQueue`. `test + advance(by:)` can. Keep the `AnyCancellable` (deinit cancels), or for `await over publisher.values` (iOS 15, `Failure == Never`).
- AsyncStream**: drive the SUT through `AsyncStream.makeStream(of:)` (5.9) — the test holds the continuation. Consume *bounded*: `s.prefix(3)` or `finish()`; an unfinished stream never ends the `for await`.

Ordering & cancellation

- `Task { await vm.load() }` then assert: the next line usually runs before the task body. `await task.value`, call `await vm.load()` directly, or use `Task.immediate` (6.2: synchronous up to its first suspension). `withMainSerialExecutor` (swift-concurrency-extras) serialises a test's tasks — tests only, flips a global.
- `CancellationError` is promised by `Task.checkCancellation()` and `Task.sleep`, not by every API (`URLSession: URLError`). Assert the outcome *and* the side effect that must not happen. Swift Testing: a `CancellationError` in a cancelled test = cancelled, not failed; `Test.cancel()` (6.3).
- Cleanup (6.4): `defer { await ... }` runs to completion; `withTaskCancellationShield` hides cancellation from it (no back-deployment).

Remember

Await the event, never sleep. Stub bytes, not objects — the real status check and decoder run. Four failures: wire · status · decode · cancel. Time is a dependency: inject it, advance it.